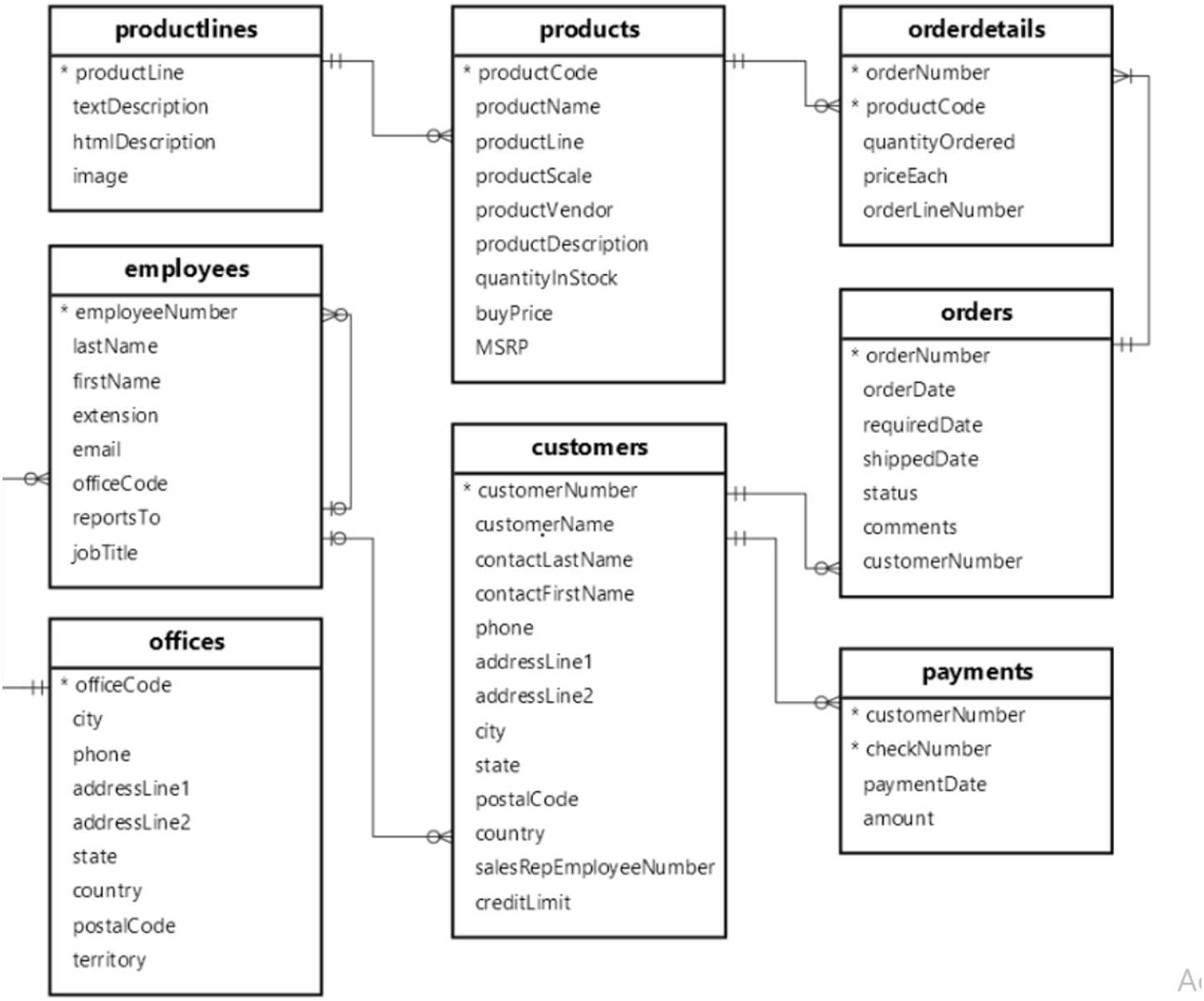

The fields above mentioned in the * are the Primary keys of that table
eg: *officeCode, *CustomerNumber etc.

. CREATE TABLES using the below structure and insert data into it

```
-- OFFICES
CREATE TABLE offices (
   officeCode VARCHAR(10) PRIMARY KEY,
   city VARCHAR(50),
   phone VARCHAR(50),
   addressLine1 VARCHAR(100),
   addressLine2 VARCHAR(100),
   state VARCHAR(50),
   country VARCHAR(50),
   postalCode VARCHAR(15),
```

```
    territory VARCHAR(50)
);

-- EMPLOYEES
CREATE TABLE employees (
    employeeNumber INT PRIMARY KEY,
    lastName VARCHAR(50),
    firstName VARCHAR(50),
    extension VARCHAR(10),
    email VARCHAR(100),
    officeCode VARCHAR(10),
    reportsTo INT,
    jobTitle VARCHAR(50),
    FOREIGN KEY (officeCode) REFERENCES offices(officeCode),
    FOREIGN KEY (reportsTo) REFERENCES employees(employeeNumber)
);

-- CUSTOMERS
CREATE TABLE customers (
    customerNumber INT PRIMARY KEY,
    customerName VARCHAR(100),
    contactLastName VARCHAR(50),
    contactFirstName VARCHAR(50),
    phone VARCHAR(50),
    addressLine1 VARCHAR(100),
    addressLine2 VARCHAR(100),
    city VARCHAR(50),
    state VARCHAR(50),
    postalCode VARCHAR(15),
    country VARCHAR(50),
    salesRepEmployeeNumber INT,
    creditLimit DECIMAL(10,2),
    FOREIGN KEY (salesRepEmployeeNumber) REFERENCES employees(employeeNumber)
);

-- PRODUCTLINES
CREATE TABLE productlines (
    productLine VARCHAR(50) PRIMARY KEY,
    textDescription TEXT,
    htmlDescription TEXT,
    image TEXT
);

-- PRODUCTS
CREATE TABLE products (
    productCode VARCHAR(15) PRIMARY KEY,
```

```
    productName VARCHAR(100),
    productLine VARCHAR(50),
    productScale VARCHAR(10),
    productVendor VARCHAR(50),
    productDescription TEXT,
    quantityInStock INT,
    buyPrice DECIMAL(10,2),
    MSRP DECIMAL(10,2),
    FOREIGN KEY (productLine) REFERENCES productlines(productLine)
);

-- ORDERS
CREATE TABLE orders (
    orderNumber INT PRIMARY KEY,
    orderDate DATE,
    requiredDate DATE,
    shippedDate DATE,
    status VARCHAR(20),
    comments TEXT,
    customerNumber INT,
    FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);

-- ORDERDETAILS
CREATE TABLE orderdetails (
    orderNumber INT,
    productCode VARCHAR(15),
    quantityOrdered INT,
    priceEach DECIMAL(10,2),
    orderLineNumber INT,
    PRIMARY KEY (orderNumber, productCode),
    FOREIGN KEY (orderNumber) REFERENCES orders(orderNumber),
    FOREIGN KEY (productCode) REFERENCES products(productCode)
);

-- PAYMENTS
CREATE TABLE payments (
    customerNumber INT,
    checkNumber VARCHAR(50),
    paymentDate DATE,
    amount DECIMAL(10,2),
    PRIMARY KEY (customerNumber, checkNumber),
    FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
);
```

INSERT SAMPLE DATA

Offices

```
INSERT INTO offices VALUES
('1', 'New York', '1234567890', '5th Ave', NULL, 'NY', 'USA', '10001', 'NA'),
('2', 'London', '9876543210', 'Baker St', NULL, NULL, 'UK', 'NW1', 'EMEA');
```

Employees

```
INSERT INTO employees VALUES
(1001, 'Smith', 'John', 'x101', 'john@company.com', '1', NULL, 'Manager'),
(1002, 'Doe', 'Jane', 'x102', 'jane@company.com', '1', 1001, 'Sales Rep'),
(1003, 'Brown', 'Mike', 'x103', 'mike@company.com', '2', 1001, 'Sales Rep');
```

Customers

```
INSERT INTO customers VALUES
(2001, 'ABC Corp', 'Lee', 'Chris', '1111111111', 'Street 1', NULL, 'NYC', 'NY', '10001', 'USA', 1002, 50000),
(2002, 'XYZ Ltd', 'Kim', 'Pat', '2222222222', 'Street 2', NULL, 'London', NULL, 'NW1', 'UK', 1003, 70000),
(2003, 'Global Inc', 'Singh', 'Raj', '3333333333', 'Street 3', NULL, 'Delhi', NULL, '110001', 'India', 1002, 60000);
```

Product Lines

```
INSERT INTO productlines VALUES
('Cars', 'Car models', NULL, NULL),
('Bikes', 'Bike models', NULL, NULL);
```

Products

```
INSERT INTO products VALUES
('P101', 'Toyota Camry', 'Cars', '1:18', 'Toyota', 'Sedan model', 100, 20.00, 30.00),
('P102', 'Honda Civic', 'Cars', '1:18', 'Honda', 'Compact model', 150, 18.00, 28.00),
('P201', 'Yamaha Bike', 'Bikes', '1:12', 'Yamaha', 'Sport bike', 200, 10.00, 15.00);
```

Orders

```
INSERT INTO orders VALUES
(3001, '2024-01-01', '2024-01-05', '2024-01-03', 'Shipped', NULL, 2001),
(3002, '2024-02-01', '2024-02-05', '2024-02-04', 'Shipped', NULL, 2002),
(3003, '2024-03-01', '2024-03-06', NULL, 'Pending', NULL, 2003);
```

Order Details

```
INSERT INTO orderdetails VALUES
(3001, 'P101', 10, 30.00, 1),
```

(3001, 'P201', 5, 15.00, 2),
(3002, 'P102', 7, 28.00, 1),
(3003, 'P101', 3, 30.00, 1);

Payments

INSERT INTO payments VALUES
(2001, 'CHK001', '2024-01-10', 300.00),
(2002, 'CHK002', '2024-02-10', 500.00),
(2003, 'CHK003', '2024-03-10', 150.00);

EASY LEVEL (1–20)

1. List all customers ordered by customerName.

```
mysql>  select * from customers order by customername asc;
+----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
| customerNumber | customerName | contactLastName | contactFirstName | phone      | addressLine1 | addressLine2 | city   | state | postalCode | country | sa
lesRepEmployeeNumber | creditLimit |
+----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
|           2001 | ABC Corp     | Lee             | Chris            | 1111111111 | Street 1     | NULL         | NYC    | NY    | 10001      | USA     |
                1002 |    50000.00 |
|           2003 | Global Inc   | Singh           | Raj              | 3333333333 | Street 3     | NULL         | Delhi  | NULL  | 110001     | India   |
                1002 |    60000.00 |
|           2002 | XYZ Ltd      | Kim             | Pat              | 2222222222 | Street 2     | NULL         | London | NULL  | NW1        | UK      |
                1003 |    70000.00 |
+----------------+--------------+-----------------+------------------+------------+--------------+--------------+--------+-------+------------+---------+---
--------------------+-------------+
3 rows in set (0.00 sec)
```

2. Display all products ordered by buyPrice descending.

```
mysql> select * from products order by buyprice desc;
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
| productCode | productName  | productLine | productScale | productVendor | productDescription | quantityInStock | buyPrice | MSRP  |
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
| P101        | Toyota Camry | Cars        | 1:18         | Toyota        | Sedan model        |             100 |    20.00 | 30.00 |
| P102        | Honda Civic  | Cars        | 1:18         | Honda         | Compact model      |             150 |    18.00 | 28.00 |
| P201        | Yamaha Bike  | Bikes       | 1:12         | Yamaha        | Sport bike         |             200 |    10.00 | 15.00 |
+-------------+--------------+-------------+--------------+---------------+--------------------+-----------------+----------+-------+
3 rows in set (0.00 sec)
```

3. Show all employees ordered by lastName.

```
mysql> select * from employees order by lastName asc;
+----------------+----------+-----------+-----------+------------------+-------------+-----------+-----------+
| employeeNumber | lastName | firstName | extension | email            | officeCode | reportsTo | jobTitle  |
+----------------+----------+-----------+-----------+------------------+-------------+-----------+-----------+
|           1003 | Brown    | Mike      | x103      | mike@company.com | 2           |      1001 | Sales Rep |
|           1002 | Doe      | Jane      | x102      | jane@company.com | 1           |      1001 | Sales Rep |
|           1001 | Smith    | John      | x101      | john@company.com | 1           |      NULL | Manager   |
+----------------+----------+-----------+-----------+------------------+-------------+-----------+-----------+
3 rows in set (0.00 sec)
```

4. List all offices sorted by country.

```
mysql> select * from offices order by country asc;
+-------------+----------+------------+--------------+--------------+-------+---------+------------+-----------+
| officeCode | city     | phone      | addressLine1 | addressLine2 | state | country | postalCode | territory |
+-------------+----------+------------+--------------+--------------+-------+---------+------------+-----------+
| 2           | London   | 9876543210 | Baker St     | NULL         | NULL  | UK      | NW1        | EMEA      |
| 1           | New York | 1234567890 | 5th Ave      | NULL         | NY    | USA     | 10001      | NA        |
+-------------+----------+------------+--------------+--------------+-------+---------+------------+-----------+
2 rows in set (0.00 sec)
```

5. Get all orders sorted by orderDate descending.

```
mysql> select * from orders order by orderDate desc;
+-------------+------------+--------------+-------------+---------+----------+----------------+
| orderNumber | orderDate  | requiredDate | shippedDate | status  | comments | customerNumber |
+-------------+------------+--------------+-------------+---------+----------+----------------+
|        3003 | 2024-03-01 | 2024-03-06   | NULL        | Pending | NULL     |           2003 |
|        3002 | 2024-02-01 | 2024-02-05   | 2024-02-04  | Shipped | NULL     |           2002 |
|        3001 | 2024-01-01 | 2024-01-05   | 2024-01-03  | Shipped | NULL     |           2001 |
+-------------+------------+--------------+-------------+---------+----------+----------------+
3 rows in set (0.00 sec)
```

6. Show customer names with their sales representative (employee name).

```
mysql> SELECT
    ->     c.customerName,
    ->     e.firstName,
    ->     e.lastName
    -> FROM customers c
    -> JOIN employees e
    -> ON c.salesRepEmployeeNumber = e.employeeNumber;
+--------------+-----------+----------+
| customerName | firstName | lastName |
+--------------+-----------+----------+
| ABC Corp     | Jane      | Doe      |
| XYZ Ltd      | Mike      | Brown    |
| Global Inc   | Jane      | Doe      |
+--------------+-----------+----------+
3 rows in set (0.05 sec)
```

7. List all products with their product line.

```
mysql> select
    ->  p.productName,
    ->  p.productLine
    -> FROM products p;
+--------------+-------------+
| productName  | productLine |
+--------------+-------------+
| Toyota Camry | Cars        |
| Honda Civic  | Cars        |
| Yamaha Bike  | Bikes       |
+--------------+-------------+
3 rows in set (0.00 sec)
```

8. Display orders with customer names.

```
mysql> select
    ->  o.orderNumber,
    ->     c.customerName
    -> FROM orders o
    -> JOIN customers c
    -> ON o.customerNumber = c.customerNumber;
+-------------+--------------+
| orderNumber | customerName |
+-------------+--------------+
|        3001 | ABC Corp     |
|        3002 | XYZ Ltd      |
|        3003 | Global Inc   |
+-------------+--------------+
3 rows in set (0.00 sec)
```

9. Show order details with product names.

```
mysql> SELECT
    ->     o.orderNumber,
    ->     p.productName,
    ->     o.quantityOrdered,
    ->     o.priceEach
    -> FROM orderdetails o
    -> JOIN products p
    -> ON o.productCode = p.productCode;
+-------------+--------------+-----------------+-----------+
| orderNumber | productName  | quantityOrdered | priceEach |
+-------------+--------------+-----------------+-----------+
|        3001 | Toyota Camry |              10 |     30.00 |
|        3003 | Toyota Camry |               3 |     30.00 |
|        3002 | Honda Civic  |               7 |     28.00 |
|        3001 | Yamaha Bike  |               5 |     15.00 |
+-------------+--------------+-----------------+-----------+
4 rows in set (0.00 sec)
```

10. List payments with customer names.

```
mysql> SELECT
    ->     c.customerName,
    ->     p.checkNumber,
    ->     p.paymentDate,
    ->     p.amount
    -> FROM payments p
    -> JOIN customers c
    -> ON p.customerNumber = c.customerNumber;
+--------------+-------------+-------------+--------+
| customerName | checkNumber | paymentDate | amount |
+--------------+-------------+-------------+--------+
| ABC Corp     | CHK001      | 2024-01-10  | 300.00 |
| XYZ Ltd      | CHK002      | 2024-02-10  | 500.00 |
| Global Inc   | CHK003      | 2024-03-10  | 150.00 |
+--------------+-------------+-------------+--------+
3 rows in set (0.00 sec)
```

11. Count total number of customers.

```
mysql> select count(*) as total_customers from customers;
+-----------------+
| total_customers |
+-----------------+
|               3 |
+-----------------+
1 row in set (0.05 sec)
```

12. Count total number of orders.

```
mysql> select count(*) as total_orders from orders;
+--------------+
| total_orders |
+--------------+
|            3 |
+--------------+
1 row in set (0.00 sec)
```

13. Find average buyPrice of products.

```
mysql> select AVG(buyprice) as Avg_Buy_Price from products;
+---------------+
| Avg_Buy_Price |
+---------------+
|     16.000000 |
+---------------+
1 row in set (0.00 sec)
```

14. Find maximum MSRP from products.

```
mysql> select MAX(MSRP) as Max_Msrp from products;
+----------+
| Max_Msrp |
+----------+
|    30.00 |
+----------+
1 row in set (0.00 sec)
```

15. Find total number of employees.

```
mysql> select count(*) As total_employees from employees;
+-----------------+
| total_employees |
+-----------------+
|               3 |
+-----------------+
1 row in set (0.00 sec)
```

16. Count customers per country.

```
mysql> select country,count(*) As Total_customers
    -> from customers
    -> group by country;
+---------+-----------------+
| country | Total_customers |
+---------+-----------------+
| USA     |               1 |
| UK      |               1 |
| India   |               1 |
+---------+-----------------+
3 rows in set (0.05 sec)
```

17. Count orders per status.

```
mysql>  select status,count(*) As total_orders
    -> from orders
    ->  group by status;
+---------+--------------+
| status  | total_orders |
+---------+--------------+
| Shipped |            2 |
| Pending |            1 |
+---------+--------------+
2 rows in set (0.00 sec)
```

18. Count products per product line.

```
mysql> select productLine,Count(*) As total_products
    -> from products
    -> group by productLine;
+-------------+----------------+
| productLine | total_products |
+-------------+----------------+
| Bikes       |              1 |
| Cars        |              2 |
+-------------+----------------+
2 rows in set (0.00 sec)
```

19. Find average credit limit per country.

```
mysql> SELECT
    ->     country,
    ->     AVG(creditLimit) AS avg_credit_limit
    -> FROM customers
    -> GROUP BY country;
+---------+------------------+
| country | avg_credit_limit |
+---------+------------------+
| USA     |      50000.000000 |
| UK      |      70000.000000 |
| India   |      60000.000000 |
+---------+------------------+
3 rows in set (0.00 sec)
```

20. Count employees per office.

```
mysql> SELECT
    ->     officeCode,
    ->     COUNT(*) AS total_employees
    -> FROM employees
    -> GROUP BY officeCode;
+------------+----------------+
| officeCode | total_employees |
+------------+----------------+
| 1          |              2 |
| 2          |              1 |
+------------+----------------+
2 rows in set (0.00 sec)
```

MEDIUM LEVEL (21–50)

21. Count number of orders per customer.

```
mysql> SELECT
    ->     customerNumber,
    ->     COUNT(*) AS total_orders
    -> FROM orders
    -> GROUP BY customerNumber;
+----------------+--------------+
| customerNumber | total_orders |
+----------------+--------------+
|           2001 |            1 |
|           2002 |            1 |
|           2003 |            1 |
+----------------+--------------+
3 rows in set (0.00 sec)
```

22. Find total payment amount per customer.

```
mysql> SELECT
    ->     customerNumber,
    ->     SUM(amount) AS total_payment
    -> FROM payments
    -> GROUP BY customerNumber;
+----------------+---------------+
| customerNumber | total_payment |
+----------------+---------------+
|           2001 |        300.00 |
|           2002 |        500.00 |
|           2003 |        150.00 |
+----------------+---------------+
3 rows in set (0.00 sec)
```

23. List total quantity ordered per product.

```
mysql> SELECT
    ->     productCode,
    ->     SUM(quantityOrdered) AS total_quantity
    -> FROM orderdetails
    -> GROUP BY productCode;
+-------------+----------------+
| productCode | total_quantity |
+-------------+----------------+
| P101        |             13 |
| P102        |              7 |
| P201        |              5 |
+-------------+----------------+
3 rows in set (0.06 sec)
```

24. Show total revenue per product (quantityOrdered * priceEach).

```
mysql> SELECT
    ->     productCode,
    ->     SUM(quantityOrdered * priceEach) AS total_revenue
    -> FROM orderdetails
    -> GROUP BY productCode;
+-------------+---------------+
| productCode | total_revenue |
+-------------+---------------+
| P101        |        390.00 |
| P102        |        196.00 |
| P201        |         75.00 |
+-------------+---------------+
3 rows in set (0.01 sec)
```

25. Find number of customers handled by each employee.

```
mysql> SELECT
    ->     salesRepEmployeeNumber,
    ->     COUNT(*) AS total_customers
    -> FROM customers
    -> GROUP BY salesRepEmployeeNumber;
+------------------------+-----------------+
| salesRepEmployeeNumber | total_customers |
+------------------------+-----------------+
|                   1002 |               2 |
|                   1003 |               1 |
+------------------------+-----------------+
2 rows in set (0.00 sec)
```

26. Find customers with more than 5 orders.

```
mysql> SELECT
    ->     customerNumber,
    ->     COUNT(orderNumber) AS total_orders
    -> FROM orders
    -> GROUP BY customerNumber
    -> having Count(orderNumber) > 5;
Empty set (0.06 sec)
```

27. List products with total quantity ordered > 100.

```
mysql> SELECT
    ->     productCode,
    ->     SUM(quantityOrdered) AS total_quantity
    -> FROM orderdetails
    -> GROUP BY productCode
    -> HAVING SUM(quantityOrdered) > 100;
Empty set (0.07 sec)
```

28. Find employees managing more than 3 customers.

```
mysql> SELECT
    ->     salesRepEmployeeNumber,
    ->     COUNT(*) AS total_customers
    -> FROM customers
    -> GROUP BY salesRepEmployeeNumber
    -> HAVING COUNT(*) > 3;
Empty set (0.03 sec)
```

29. Show countries with more than 10 customers.

```
mysql> select
    ->     country,
    ->     count(*) as total_customers
    -> from customers
    -> group by country
    -> having count(*) > 10;
Empty set (0.00 sec)
```

30. Find product lines having more than 5 products.

```
mysql> select
    ->     productline,
    ->     count(*) as total_products
    -> from products
    -> group by productline
    -> having count(*) > 5;
Empty set (0.01 sec)
```

31. List top 5 customers by total payment amount.

```
mysql> select
    ->     c.customername,
    ->     sum(p.amount) as total_payment
    -> from customers c
    -> join payments p
    -> on c.customernumber = p.customernumber
    -> group by c.customername
    -> order by total_payment desc
    -> ;
+--------------+---------------+
| customername | total_payment |
+--------------+---------------+
| XYZ Ltd      |        500.00 |
| ABC Corp     |        300.00 |
| Global Inc   |        150.00 |
+--------------+---------------+
3 rows in set (0.03 sec)
```

32. Show products ordered by total sales descending.

```
mysql> select
    ->  p.productname,
    ->     sum(o.quantityordered * o.priceeach) as total_sales
    -> from orderdetails o
    -> join products p
    -> on o.productcode = p.productcode
    -> group by p.productname
    -> order by total_sales desc;
+--------------+-------------+
| productname  | total_sales |
+--------------+-------------+
| Toyota Camry |      390.00 |
| Honda Civic  |      196.00 |
| Yamaha Bike  |       75.00 |
+--------------+-------------+
3 rows in set (0.01 sec)
```

33. List employees sorted by number of customers handled.

```
mysql> select
    ->     e.firstname,
    ->     e.lastname,
    ->     count(c.customernumber) as total_customers
    -> from employees e
    -> left join customers c
    -> on e.employeenumber = c.salesrepemployeenumber
    -> group by e.employeenumber, e.firstname, e.lastname
    -> order by total_customers desc;
+-----------+----------+-----------------+
| firstname | lastname | total_customers |
+-----------+----------+-----------------+
| Jane      | Doe      |               2 |
| Mike      | Brown    |               1 |
| John      | Smith    |               0 |
+-----------+----------+-----------------+
3 rows in set (0.00 sec)
```

34. Show countries sorted by total customers.

```
mysql> select
    ->     country,
    ->     count(*) as total_customers
    -> from customers
    -> group by country
    -> order by total_customers desc;
+---------+-----------------+
| country | total_customers |
+---------+-----------------+
| USA     |               1 |
| UK      |               1 |
| India   |               1 |
+---------+-----------------+
3 rows in set (0.00 sec)
```

35. Display top 10 most ordered products.

```
mysql> select
    ->     p.productname,
    ->     sum(o.quantityordered) as total_quantity
    -> from orderdetails o
    -> join products p
    -> on o.productcode = p.productcode
    -> group by p.productname
    -> order by total_quantity desc
    -> ;
+--------------+----------------+
| productname  | total_quantity |
+--------------+----------------+
| Toyota Camry |             13 |
| Honda Civic  |              7 |
| Yamaha Bike  |              5 |
+--------------+----------------+
3 rows in set (0.00 sec)
```

Multi-table JOINS

36. Get customer name, order number, and product name.

```
mysql> select
    ->     c.customername,
    ->     o.ordernumber,
    ->     p.productname
    -> from customers c
    -> join orders o
    -> on c.customernumber = o.customernumber
    -> join orderdetails od
    -> on o.ordernumber = od.ordernumber
    -> join products p
    -> on od.productcode = p.productcode;
+--------------+-------------+--------------+
| customername | ordernumber | productname  |
+--------------+-------------+--------------+
| ABC Corp     |        3001 | Toyota Camry |
| ABC Corp     |        3001 | Yamaha Bike  |
| XYZ Ltd      |        3002 | Honda Civic  |
| Global Inc   |        3003 | Toyota Camry |
+--------------+-------------+--------------+
4 rows in set (0.00 sec)
```

37. Show employee name, customer name, and total payments.

```
mysql> select
    ->     e.firstname,
    ->     e.lastname,
    ->     c.customername,
    ->     sum(p.amount) as total_payment
    -> from employees e
    -> join customers c
    -> on e.employeenumber = c.salesrepemployeenumber
    -> join payments p
    -> on c.customernumber = p.customernumber
    -> group by e.employeenumber, e.firstname, e.lastname, c.customername;
+-----------+----------+--------------+---------------+
| firstname | lastname | customername | total_payment |
+-----------+----------+--------------+---------------+
| Jane      | Doe      | ABC Corp     |        300.00 |
| Jane      | Doe      | Global Inc   |        150.00 |
| Mike      | Brown    | XYZ Ltd      |        500.00 |
+-----------+----------+--------------+---------------+
3 rows in set (0.00 sec)
```

38. List order details with product line and quantity ordered.

```
mysql> select
    -> o.ordernumber,
    ->     p.productline,
    ->     o.quantityordered
    -> from orderdetails o
    -> join products p
    -> on o.productcode = p.productcode;
+-------------+-------------+-----------------+
| ordernumber | productline | quantityordered |
+-------------+-------------+-----------------+
|        3001 | Bikes       |               5 |
|        3001 | Cars        |              10 |
|        3003 | Cars        |               3 |
|        3002 | Cars        |               7 |
+-------------+-------------+-----------------+
4 rows in set (0.00 sec)
```

39. Display office city with employee count.

```
mysql> select
    ->     o.city,
    ->     count(e.employeenumber) as total_employees
    -> from offices o
    -> left join employees e
    -> on o.officecode = e.officecode
    -> group by o.city;
+----------+----------------+
| city     | total_employees |
+----------+----------------+
| New York |               2 |
| London   |               1 |
+----------+----------------+
2 rows in set (0.00 sec)
```

40. Show product vendor with total quantity sold.

```
mysql> select
    ->     p.productvendor,
    ->     sum(o.quantityordered) as total_quantity
    -> from products p
    -> join orderdetails o
    -> on p.productcode = o.productcode
    -> group by p.productvendor;
+---------------+----------------+
| productvendor | total_quantity |
+---------------+----------------+
| Toyota        |             13 |
| Honda         |              7 |
| Yamaha        |              5 |
+---------------+----------------+
3 rows in set (0.00 sec)
```

Advanced Aggregations

41. Find average order value per customer.

```
mysql> select
    ->     c.customername,
    ->     avg(order_total) as avg_order_value
    -> from customers c
    -> join (
    ->     select
    ->         o.ordernumber,
    ->         o.customernumber,
    ->         sum(od.quantityordered * od.priceeach) as order_total
    ->     from orders o
    ->     join orderdetails od
    ->     on o.ordernumber = od.ordernumber
    ->     group by o.ordernumber, o.customernumber
    -> ) as order_values
    -> on c.customernumber = order_values.customernumber
    -> group by c.customername;
+--------------+-----------------+
| customername | avg_order_value |
+--------------+-----------------+
| ABC Corp     |      375.000000 |
| XYZ Ltd      |      196.000000 |
| Global Inc   |       90.000000 |
+--------------+-----------------+
3 rows in set (0.01 sec)
```

42. Show total revenue generated per country.

```
mysql> select
    ->     c.country,
    ->     sum(od.quantityordered * od.priceeach) as total_revenue
    -> from customers c
    -> join orders o
    -> on c.customernumber = o.customernumber
    -> join orderdetails od
    -> on o.ordernumber = od.ordernumber
    -> group by c.country;
+---------+---------------+
| country | total_revenue |
+---------+---------------+
| USA     |        375.00 |
| UK      |        196.00 |
| India   |         90.00 |
+---------+---------------+
3 rows in set (0.00 sec)
```

43. Find customers whose total payments exceed 50,000.

```
mysql> select
    ->     c.customername,
    ->     sum(p.amount) as total_payment
    -> from customers c
    -> join payments p
    -> on c.customernumber = p.customernumber
    -> group by c.customername
    -> having sum(p.amount) > 50000;
Empty set (0.05 sec)
```

44. List products where average order quantity > 50.

```
mysql> select
    ->     p.productname,
    ->     avg(o.quantityordered) as avg_quantity
    -> from orderdetails o
    -> join products p
    -> on o.productcode = p.productcode
    -> group by p.productname
    -> having avg(o.quantityordered) > 50;
Empty set (0.00 sec)
```

45. Find top 3 product lines by total revenue.

```
mysql> select
    ->     p.productline,
    ->     sum(od.quantityordered * od.priceeach) as total_revenue
    -> from products p
    -> join orderdetails od
    -> on p.productcode = od.productcode
    -> group by p.productline
    -> order by total_revenue desc
    -> ;
+-------------+---------------+
| productline | total_revenue |
+-------------+---------------+
| Cars        |        586.00 |
| Bikes       |         75.00 |
+-------------+---------------+
2 rows in set (0.04 sec)
```

46. Show customers with credit limit > average credit limit.

```
mysql> select
    ->     customername,
    ->     creditlimit
    -> from customers
    -> where creditlimit > (
    ->     select avg(creditlimit)
    ->     from customers
    -> );
+---------------+--------------+
| customername | creditlimit |
+---------------+--------------+
| XYZ Ltd       |     70000.00 |
+---------------+--------------+
1 row in set (0.00 sec)
```

47. Find orders where total amount > 10,000.

```
mysql> select
    ->     o.ordernumber,
    ->     sum(od.quantityordered * od.priceeach) as total_amount
    -> from orders o
    -> join orderdetails od
    -> on o.ordernumber = od.ordernumber
    -> group by o.ordernumber
    -> having sum(od.quantityordered * od.priceeach) > 10000;
Empty set (0.00 sec)
```

48. List employees working in offices with more than 5 employees.

```
mysql> select
    ->     firstname,
    ->     lastname,
    ->     officecode
    -> from employees
    -> where officecode in (
    ->     select officecode
    ->     from employees
    ->     group by officecode
    ->     having count(*) > 5
    -> );
Empty set (0.03 sec)
```

49. Find products with stock less than average stock.

```
mysql> select
    ->     productname,
    ->     quantityinstock
    -> from products
    -> where quantityinstock < (
    ->     select avg(quantityinstock)
    ->     from products
    -> );
+--------------+-----------------+
| productname  | quantityinstock |
+--------------+-----------------+
| Toyota Camry |             100 |
+--------------+-----------------+
1 row in set (0.00 sec)
```